REPORT

Reviewer report: pallets/itsdangerous#428: require

padding when decoding base64 on Python 3.15+

Run itsdangerous-428, tested 2026-11-09 08:15 IST by ops/daily_run.py (route source-pair). Proof-of-Change

commit ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 11:59 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

NEEDS ATTENTION

Recommended action: Reject Confidence: High

The run is trustworthy and found a difference a person should look at. Every check passed and every claim cites

a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: require padding when decoding base64 on Python 3.15+ (pallets/itsdangerous#428).

• What we observed: probe: 5 observation(s) need attention: c1.padded_valid does not differ although the claim

says it does (C1); c1.unpadded_valid does not differ although the claim says it does (C1);

c2.malformed_no_padding does not differ although the claim says it does (C2); c3.empty_input does not differ

although the claim says it does (C3); tests: 8 test(s) ran on main, 8 on the PR; 0 new test(s) pass, 8 unchanged.

• What you should do: Do not approve as is. Read 'What changed in behavior' and decide with the PR author.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 8.9 GB (of 15.6 GB).

• Setup check: free disk 93.1 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: itsdangerous python setup install instructions https://github.com/pallets/itsdangerous site:github.com/pallets/itsdangerous

• Query: itsdangerous python documentation install https://github.com/pallets/itsdangerous site:github.com/pallets/itsdangerous

• Query: itsdangerous python install requirements https://pypi.org/project/itsdangerous/ https://github.com/pallets/itsdangerous site:pypi.org

• Source: itsdangerous/pyproject.toml at main · pallets/itsdangerous · GitHub — https://github.com/pallets/itsdangerous/blob/main/pyproject.toml

• Source: itsdangerous · PyPI — https://pypi.org/project/itsdangerous/

• Source: Releases · pallets/itsdangerous · GitHub — https://github.com/pallets/itsdangerous/releases

• Source: itsdangerous/CHANGES.rst at main · pallets/itsdangerous · GitHub — https://github.com/pallets/itsdangerous/blob/main/CHANGES.rst

• Source: itsdangerous/src/itsdangerous/__init__.py at main · pallets/itsdangerous · GitHub — https://github.com/pallets/itsdangerous/blob/main/src/itsdangerous/__init__.py

• Setup plan: the same packages on main and pr.

• Setup plan: install freezegun (listed in the tests extra) on main and pr.

• Setup plan: install pytest (listed in the tests extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10, >=3.8

• Installed freezegun 1.5.5 from pyproject (listed in the tests extra) on main and pr.

• Installed pytest 9.2.0 from pyproject (listed in the tests extra) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (8 unchanged, 3 runs per side) | pytest-pr-1.log:7 «8 passed in 0.49s»

| P-C1 | On Python 3.15+, urlsafe_b64decode requires proper padding and will reject inputs without it. (PR

description) | not observed | probe model-p1, c1.padded_valid: same on both sides: "abc" |

probe-model-p1-pr-1.jsonl:1 «"observation": "c1.padded_valid"»

| P-C2 | urlsafe_b64decode will raise BadData for malformed input lacking padding on Python 3.15+. (PR

description) | not observed | probe model-p2, c2.malformed_no_padding: same on both sides: "BadData" |

probe-model-p2-pr-1.jsonl:1 «"observation": "c2.malformed_no_padding"»

| P-C3 | The PR resolves the test failures in the 'bad payload' tests when using zlib-ng as the compression library.

(PR description) | not observed | probe model-p3, c3.empty_input: same on both sides: "" |

probe-model-p3-pr-1.jsonl:1 «"observation": "c3.empty_input"»

Citation check (PASS): 4 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 8 / 8 | | Tests unchanged | 8 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | -

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen 8083a3ea2640

at 2026-11-09T02:47:18.341646+00:00): base64_decode padding requirement

• P-C1 (PR description): On Python 3.15+, urlsafe_b64decode requires proper padding and will reject inputs

without it. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.padded_valid | P-C1 | no change although the claim says it changes | same on both sides: "abc" |

probe-model-p1-main-1.jsonl:1 «"observation": "c1.padded_valid"»
probe-model-p1-pr-1.jsonl:1

«"observation": "c1.padded_valid"»

| c1.unpadded_valid | P-C1 | no change although the claim says it changes | same on both sides: "abc" |

probe-model-p1-main-1.jsonl:2 «"observation": "c1.unpadded_valid"»
probe-model-p1-pr-1.jsonl:2

«"observation": "c1.unpadded_valid"»

model-p2 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p2.py, frozen abdd75fab4f6 at

2026-11-09T02:47:18.344576+00:00): base64_decode BadData on malformed input lacking padding

• P-C2 (PR description): urlsafe_b64decode will raise BadData for malformed input lacking padding on Python

3.15+. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c2.malformed_no_padding | P-C2 | no change although the claim says it changes | same on both sides:

"BadData" | probe-model-p2-main-1.jsonl:1 «"observation":

"c2.malformed_no_padding"»
probe-model-p2-pr-1.jsonl:1 «"observation": "c2.malformed_no_padding"»

model-p3 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p3.py, frozen 18a7fd50a024

at 2026-11-09T02:47:18.344713+00:00): base64_decode behavior for empty and invalid padded inputs

• P-C3 (PR description): The PR resolves the test failures in the 'bad payload' tests when using zlib-ng as the

compression library. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c3.empty_input | P-C3 | no change although the claim says it changes | same on both sides: "" |

probe-model-p3-main-1.jsonl:1 «"observation": "c3.empty_input"»
probe-model-p3-pr-1.jsonl:1

«"observation": "c3.empty_input"»

| c3.invalid_char_padded | P-C3 | no change although the claim says it changes | same on both sides: "ab" |

probe-model-p3-main-1.jsonl:2 «"observation": "c3.invalid_char_padded"»
probe-model-p3-pr-1.jsonl:2

«"observation": "c3.invalid_char_padded"»

diff-api (diff (auto-derived from base-side code, model-free); informational under baseline full,

diff-probes/diff-api.py, frozen f3e1bb07f6c2 at 2026-11-09T02:47:18.345680+00:00): Public functions touched by

the PR, called with type-appropriate inputs on both sides.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| itsdangerous.encoding.base64_decode.input0 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:1 «"observation":

"itsdangerous.encoding.base64_decode.input0"»
probe-diff-api-pr-1.jsonl:1 «"observation":

"itsdangerous.encoding.base64_decode.input0"»

| itsdangerous.encoding.base64_decode.input1 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:2 «"observation":

"itsdangerous.encoding.base64_decode.input1"»
probe-diff-api-pr-1.jsonl:2 «"observation":

"itsdangerous.encoding.base64_decode.input1"»

| itsdangerous.encoding.base64_decode.input10 | - | probe could not run | same on both sides: {"raises":

"BadData"} | probe-diff-api-main-1.jsonl:3 «"observation":

"itsdangerous.encoding.base64_decode.input10"»
probe-diff-api-pr-1.jsonl:3 «"observation":

"itsdangerous.encoding.base64_decode.input10"»

| itsdangerous.encoding.base64_decode.input11 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:4 «"observation":

"itsdangerous.encoding.base64_decode.input11"»
probe-diff-api-pr-1.jsonl:4 «"observation":

"itsdangerous.encoding.base64_decode.input11"»

| itsdangerous.encoding.base64_decode.input2 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:5 «"observation":

"itsdangerous.encoding.base64_decode.input2"»
probe-diff-api-pr-1.jsonl:5 «"observation":

"itsdangerous.encoding.base64_decode.input2"»

| itsdangerous.encoding.base64_decode.input3 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:6 «"observation":

"itsdangerous.encoding.base64_decode.input3"»
probe-diff-api-pr-1.jsonl:6 «"observation":

"itsdangerous.encoding.base64_decode.input3"»

| itsdangerous.encoding.base64_decode.input4 | - | same on both sides | same on both sides: {"repr": "b'i\\xb7'",

"type": "bytes"} | probe-diff-api-main-1.jsonl:7 «"observation":

"itsdangerous.encoding.base64_decode.input4"»
probe-diff-api-pr-1.jsonl:7 «"observation":

"itsdangerous.encoding.base64_decode.input4"»

| itsdangerous.encoding.base64_decode.input5 | - | probe could not run | same on both sides: {"raises":

"BadData"} | probe-diff-api-main-1.jsonl:8 «"observation":

"itsdangerous.encoding.base64_decode.input5"»
probe-diff-api-pr-1.jsonl:8 «"observation":

"itsdangerous.encoding.base64_decode.input5"»

| itsdangerous.encoding.base64_decode.input6 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:9 «"observation":

"itsdangerous.encoding.base64_decode.input6"»
probe-diff-api-pr-1.jsonl:9 «"observation":

"itsdangerous.encoding.base64_decode.input6"»

| itsdangerous.encoding.base64_decode.input7 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:10 «"observation":

"itsdangerous.encoding.base64_decode.input7"»
probe-diff-api-pr-1.jsonl:10 «"observation":

"itsdangerous.encoding.base64_decode.input7"»

| itsdangerous.encoding.base64_decode.input8 | - | probe could not run | same on both sides: {"raises":

"TypeError"} | probe-diff-api-main-1.jsonl:11 «"observation":

"itsdangerous.encoding.base64_decode.input8"»
probe-diff-api-pr-1.jsonl:11 «"observation":

"itsdangerous.encoding.base64_decode.input8"»

| itsdangerous.encoding.base64_decode.input9 | - | same on both sides | same on both sides: {"repr": "b"",

"type": "bytes"} | probe-diff-api-main-1.jsonl:12 «"observation":

"itsdangerous.encoding.base64_decode.input9"»
probe-diff-api-pr-1.jsonl:12 «"observation":

"itsdangerous.encoding.base64_decode.input9"»

Citation check (PASS): 34 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (1.3 s run).

Coverage gate (python, confidence normal): 3 of 4 changed executable line(s) ran (75%).

| Changed file | Changed executable lines | Of those, ran

| src/itsdangerous/encoding.py | 4 | 3

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested d9f3b250e0ca of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 3 of 4 changed executable line(s) ran (75%).

• Probes ran on both sides: 4 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 4 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 34 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Behavior difference found | finding | probe: 5 observation(s) need attention: c1.padded_valid does not differ

although the claim says it does (C1); c1.unpadded_valid does not differ although the claim says it does (C1);

c2.malformed_no_padding does not differ although the claim says it does (C2); c3.empty_input does not differ

although the claim says it does (C3); tests: 8 test(s) ran on main, 8 on the PR; 0 new test(s) pass, 8 unchanged |

This is about the PR, not the run: read the failing tests and their log lines and the Probes section, then decide.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 3 of 4 changed executable line(s) ran (75%)

| Probes ran on both sides | PASS | 4 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 4 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 34 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-08/itsdangerous-428 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/pallets/itsdangerous/pull/428

Evidence links

• Pull request pallets/itsdangerous#428

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-p1-main-1.jsonl

• probe-model-p1-pr-1.jsonl

• probe-model-p2-main-1.jsonl

• probe-model-p2-pr-1.jsonl

• probe-model-p3-main-1.jsonl

• probe-model-p3-pr-1.jsonl

• probe-diff-api-main-1.jsonl

• probe-diff-api-pr-1.jsonl

• main run 1 result: pytest-main-1.log:7 «8 passed in 0.49s»

• pr run 1 result: pytest-pr-1.log:7 «8 passed in 0.49s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 11:59 IST